

AI Ticket Classifier Project Report

AI-Powered Ticket Classification System Using FastAPI, Retrieval-Augmented Generation (RAG), and Large Language Models

Submitted By Name: Abhinaya Somishetty

Degree: Bachelor of Engineering (Information Technology)

Abstract

The AI Ticket Classifier is a support ticket management system built to take the manual grunt work out of sorting customer tickets. Instead of relying on keyword matching, it uses AI and large language models to actually understand what a customer is asking about, so tickets get categorized based on meaning rather than surface-level text.

Under the hood, it combines Retrieval-Augmented Generation (RAG) with ChromaDB as the vector database and Ollama as the LLM, which lets the system pull up similar past tickets and use that context to make more accurate classifications. FastAPI handles the REST API layer, PostgreSQL stores all the ticket data, and the whole thing runs in Docker containers. Prometheus and Grafana keep an eye on performance, while GitHub Actions and Docker Hub handle the CI/CD side of things — so the project isn't just a proof of concept, it's built to actually run in production.

1. Introduction

Any support team dealing with a decent volume of tickets knows the pain: hundreds or thousands come in every day, and someone has to read through them and figure out where they belong. Doing this by hand is slow, inconsistent, and honestly not a great use of people's time.

The AI Ticket Classifier tackles this by using semantic search and AI to read a ticket, find similar tickets from the past, and predict the right category and priority for it automatically.

Beyond the AI piece, the project also doubles as a demonstration of solid engineering practices — Docker containerization, monitoring, and an automated CI/CD pipeline are all part of the build, not afterthoughts.

2. Problem Statement

Most traditional ticket systems still lean on manual classification, and that comes with a familiar set of headaches:

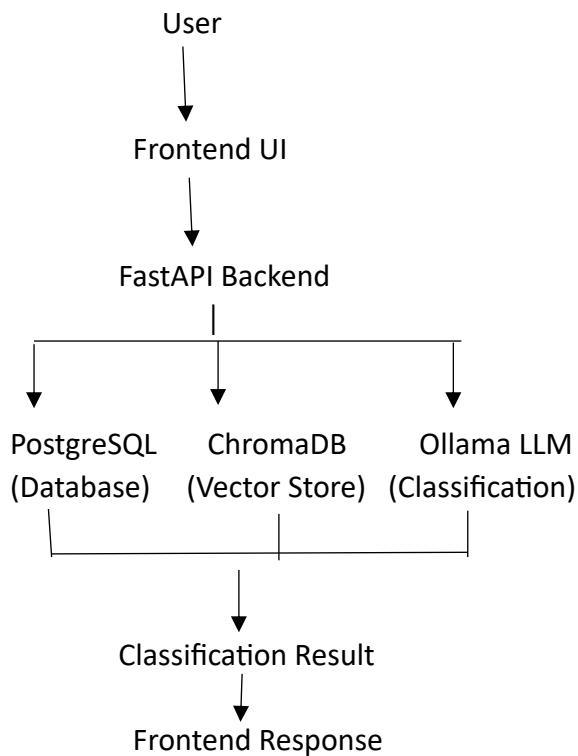
- Tickets take longer to get to the right team
- Categorization varies depending on who's doing it
- Support costs go up as volume grows
- Customers end up waiting longer and getting less consistent service
- The whole process doesn't scale well

This project sets out to automate that classification step without sacrificing accuracy.

3. Objectives

- Automate the classification of customer tickets
- Cut down the manual workload for support staff
- Improve how accurately tickets get routed
- Use embeddings to power semantic search
- Bring LLMs into the prediction pipeline
- Build REST APIs that can scale
- Containerize everything with Docker
- Monitor the app's health with Prometheus and Grafana
- Automate builds and deployments through GitHub Actions and Docker Hub

4. System Architecture



Monitoring

FastAPI → Prometheus → Grafana

CI/CD

Developer → git push → GitHub Actions → Build Docker Image → Docker Hub

5. Technologies Used

Technology	Purpose
Python 3.11	Programming Language
FastAPI	REST API Framework
PostgreSQL	Relational Database
SQLAlchemy	ORM
ChromaDB	Vector Database
Ollama	Large Language Model
Pydantic	Data Validation

Docker Containerization

Docker Compose	Multi-container Management
GitHub Actions	CI/CD Pipeline
Docker Hub	Image Registry
Prometheus	Metrics Collection
Grafana	Monitoring Dashboard
JWT	User Authentication

6. Project Modules

6.1 Authentication Module Handles user registration, login, JWT-based authentication, password hashing, and making sure API access stays secure.

6.2 Ticket Classification Module This is the core of the system — it receives incoming tickets, validates the input, generates embeddings, searches ChromaDB for similar tickets, and produces an AI-driven classification for each one.

6.3 Embedding Module Turns ticket text into numerical vectors, which is what makes semantic similarity search possible instead of just matching keywords.

6.4 Vector Search Module Uses ChromaDB to store those embeddings, retrieve similar tickets, and feed useful context to the LLM when it's making a prediction.

6.5 AI Classification Module This is where Ollama does its job — analyzing the ticket content, predicting a category, assigning a priority, and generating a confidence score for the prediction.

6.6 Database Module Stores user data, ticket details, categories, and a history of past classifications.

6.7 Monitoring Module Prometheus tracks request metrics, response times, and general API performance, and Grafana turns that data into dashboards you can actually read at a glance.

6.8 CI/CD Module GitHub Actions installs dependencies, checks that the project imports cleanly, builds the Docker image, and pushes it to Docker Hub — all without anyone needing to do it by hand.

7. Project Workflow

1. A user submits a support ticket
2. FastAPI validates the request
3. The ticket text is converted into embeddings
4. ChromaDB retrieves similar past tickets
5. That retrieved context gets passed to Ollama
6. Ollama predicts the category and priority
7. The result is saved to PostgreSQL
8. The response goes back to the frontend
9. Prometheus collects metrics on the whole process
10. Grafana visualizes that monitoring data

8. API Endpoints

Method	Endpoint	Description
POST	/auth/register	Register User
POST	/auth/login	User Login
POST	/tickets/create	Create Ticket
GET	/tickets	Retrieve Tickets
POST	/tickets/{id}	search knowledge
GET	/docs	Swagger Documentation

9. CI/CD Pipeline

GitHub Actions handles the full Continuous Integration and Delivery process, running through these steps on every push:

1. Checkout the repository
2. Install dependencies
3. Verify the Python version
4. Check code formatting
5. Build the Docker image
6. Authenticate with Docker Hub
7. Push the Docker image

The result is that every code change gets built and published automatically, without manual intervention.

10. Advantages

- Smarter, more accurate ticket classification
- Less manual work for support teams
- Faster turnaround on tickets
- Semantic search powered by embeddings
- Fully containerized deployment
- Automated CI/CD
- Real-time monitoring
- Built with production use in mind

11. Limitations

- Depends on an Ollama server running locally
- Classification accuracy is tied to the quality of the embedding model and LLM used
- A GPU helps with inference speed, though it isn't strictly required
- Currently only handles a limited set of ticket categories

12. Future Enhancements

- Deploy to AWS EC2
- Move to Kubernetes for orchestration
- Add multi-language support
- Add email notifications
- Introduce Redis caching
- Add Role-Based Access Control (RBAC)
- Support multiple LLMs
- Automate cloud deployment further
- Build out an analytics dashboard
- Extend support to a mobile application

13. Results

By the end of the project, the system was able to:

- Automatically classify support tickets
- Use ChromaDB for semantic search
- Rely on Ollama for AI-powered predictions
- Store all ticket data in PostgreSQL
- Expose REST APIs through FastAPI
- Run fully containerized in Docker
- Monitor itself with Prometheus and Grafana
- Build and deploy automatically through GitHub Actions and Docker Hub

Altogether, it shows a complete, end-to-end workflow for AI-powered ticket classification — one that's realistic enough to work in an actual production setting.

14. Conclusion

The AI Ticket Classifier brings together FastAPI, RAG, vector databases, LLMs, Docker, monitoring, and CI/CD into one cohesive, production-oriented application. By automating the classification process, it makes support operations faster and more consistent while cutting down on manual work. Just as importantly, the project reflects solid engineering and DevOps practices throughout, making it a strong example of what real-world AI application development looks like.

References

- [FastAPI Documentation](#)
- [PostgreSQL Documentation](#)
- [ChromaDB Documentation](#)
- [Ollama Documentation](#)
- [SQLAlchemy Documentation](#)
- [Docker Documentation](#)
- [Prometheus Documentation](#)
- [Grafana Documentation](#)
- [GitHub Actions Documentation](#)
- [Python 3.11 Documentation](#)